МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 6

з дисципліни «Алгоритмізація та програмування»
на тему «Реалізація алгоритмів обробки одновимірних масивів мовою С ++»

ХАІ.301. G3. 319а. 25 ЛР

Виконав студент гр. _____319а_____

_____*Андрій НОЗДРЯКОВ*___
(підпис, дата) (П.І.Б.)

Перевірив

_____ асистент Євген ПЯВКА
(підпис, дата) (П.І.Б.)

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ представлення одновимірних і масивів на мові С ++ і реалізувати декларацію, введення з консолі, обробку і виведення в консоль одновимірних масивів на мові С ++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вирішити завдання на аналіз і виведення елементів одновимірного масиву. Розмір масиву і його елементи ввести з консолі. Спершу вивести весь масив у рядок в порядку зростання індексів, потім – елементи чи підраховані результати відповідно до завдання.

Array 37:Дан масив розміру N. Знайти кількість ділянок, на яких його елементи зростають.

Завдання 2. Вирішити завдання на перетворення одновимірного масиву. Розмір масиву і його елементи ввести з консолі. Спершу вивести у консоль заданий масив, потім – змінений.

Array 90:Дан масив розміру N і ціле число K (1 ≤ K ≤ N). Видалити з масиву елемент з порядковим номером K.

Завдання 3. У функції main() організувати багаторазовий вибір одного з двох завдань. Кожне завдання описати окремою функцією без параметрів. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання

у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі *Array 37*

Вхідні дані (ім’я, опис, тип, обмеження):

N - Кількість елементів масиву, int, $2 \leq N \leq 50$;

a[i] - Елементи масиву, double, довільні дійсні числа;

Вихідні дані (ім’я, опис, тип):

result - Кількість зростаючих ділянок у масиві, int;

a[i] - Введений масив (виводиться на екран), double;

Алгоритм вирішення показано нижче/на рис. 1

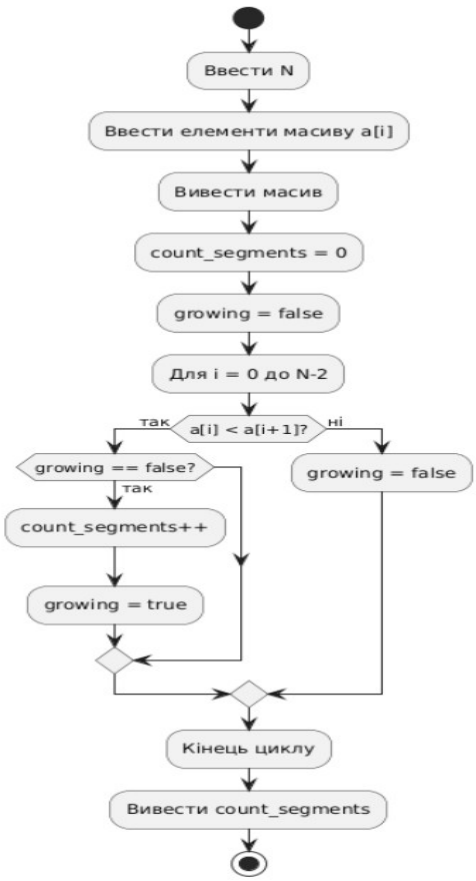

Рисунок 1 – Алгоритм вирішення Array 37

Лістинг коду вирішення задачі Array 37 наведено в дод. А (стор. 6).
Екран роботи програми показаний на рис. Б.1.

Завдання 2.

Вирішення задачі *Array 90*

Вхідні дані (ім'я, опис, тип, обмеження):

N - Кількість елементів масиву, int, $2 \leq N \leq 50$;

a[i] - Елементи масиву, double, довільні дійсні числа;

K - Номер елемента, який потрібно видалити, int, $1 \leq K \leq N$;

Вихідні дані (ім'я, опис, тип):

a[i] - Масив після видалення K-го елемента , double

Алгоритм вирішення показано нижче/на рис. 2

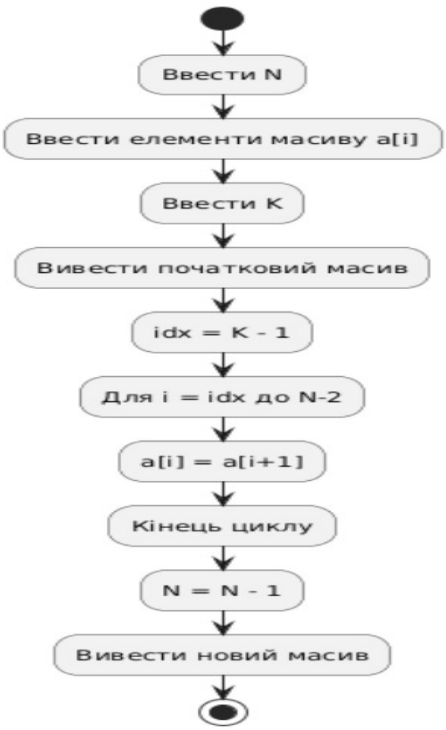

Рисунок 2 – Алгоритм вирішення Array 90

Лістинг коду вирішення задачі Array 90 наведено в дод. А (стор. 6).
Екран роботи програми показаний на рис. Б.1.

Завдання 3.

Вирішення задачі *Menu*

Вхідні дані (ім’я, опис, тип, обмеження):

choice - Номер обраного користувачем пункту меню , int, 0 ≤ choice ≤ 2 ;

Вихідні дані (ім’я, опис, тип):

Повідомлення - Відображення меню та результат виконання вибраної задачі, текст;

Алгоритм вирішення показано нижче/на рис. 3

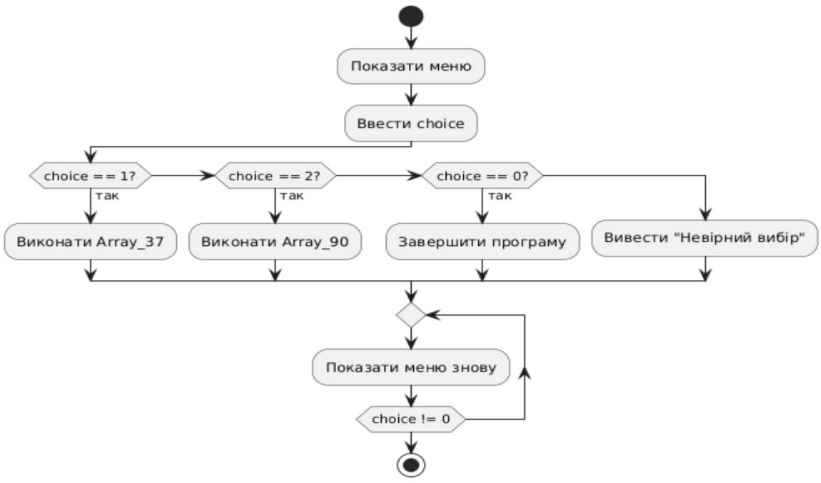

Рисунок 3 – Алгоритм вирішення Menu

Лістинг коду вирішення задачі Menu наведено в дод. А (стор. 6).

Екран роботи програми показаний на рис. Б.2.

ВИСНОВКИ

На лабораторній роботі було вивчено теоретичний матеріал з основ представлення одновимірних і масивів на мові С ++ і реалізувати декларацію, введення з консолі, обробку і виведення в консоль одновимірних масивів на мові С ++.

ДОДАТОК А

Лістинг коду програми

```
   #include <iostream>
using namespace std;

const int MAXN = 50;

// ---------------- ФУНКЦІЇ ДЛЯ МАСИВІВ ----------------

// введення масиву
void inputArray(double a[], int &N) {
    cout << "Введіть розмір масиву (2..50): ";
    cin >> N;
    cout << "Введіть " << N << " елементів масиву через пробіл: ";
    for (int i = 0; i < N; i++) {
        cin >> a[i];
    }
}

// виведення масиву
void printArray(double a[], int N) {
    for (int i = 0; i < N; i++) {
        cout << a[i] << " ";
    }
    cout << endl;
}

// підрахунок кількості зростаючих ділянок
int countIncreasingSegments(double a[], int N) {
    int count_segments = 0;
    bool growing = false;

    for (int i = 0; i < N - 1; i++) {
        if (a[i] < a[i + 1]) {
            if (!growing) {
                count_segments++;
                growing = true;
            }
        } else {
            growing = false;
        }
    }
    return count_segments;
}

// видалення елемента за номером K
void deleteElement(double a[], int &N, int K) {
    int idx = K - 1;
    for (int i = idx; i < N - 1; i++) {
        a[i] = a[i + 1];
```

```
    }
    N--;
}



// ---------------- ЗАВДАННЯ ARRAY 37 ----------------
void Array_37() {
    double a[MAXN];
    int N;

    inputArray(a, N);    // введення
    cout << "Введений масив: ";
    printArray(a, N);    // виведення

    int result = countIncreasingSegments(a, N);
    cout << "Кількість зростаючих ділянок у масиві: " << result << endl;
      cout << "(Цифра означає кількість послідовностей елементів, де кожний
наступний більший за попередній)\n";
}



// ---------------- ЗАВДАННЯ ARRAY 90 ----------------
void Array_90() {
    double a[MAXN];
    int N, K;

    inputArray(a, N);  // введення масиву
    cout << "Введіть номер елемента K для видалення (1 ≤ K ≤ " << N << "): ";
    cin >> K;

    cout << "Початковий масив: ";
    printArray(a, N);  // вивести початковий масив

    deleteElement(a, N, K);

    cout << "Масив після видалення " << K << "-го елемента: ";
    printArray(a, N);  // вивести змінений масив
}



// ---------------- ГОЛОВНЕ МЕНЮ ----------------
int main() {
    int choice;

    do {
        cout << "\nОберіть завдання:\n";
        cout << "1 - Array 37 (підрахунок зростаючих ділянок у масиві)\n";
        cout << "2 - Array 90 (видалення елемента K з масиву)\n";
```

```
        cout << "0 - Вихід\n";
        cout << "Ваш вибір: ";
        cin >> choice;

        switch (choice) {

            case 1:
                Array_37();
                break;

            case 2:
                Array_90();
                break;

            case 0:
                cout << "Програма завершена.\n";
                break;

            default:
                cout << "Невірний вибір! Спробуйте ще раз.\n";
        }

    } while (choice != 0);

    return 0;
}
```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

```
Оберіть завдання:
1 - Array 37 (підр►хунок зростаючих ділянок у масиві)
2 - Array 90 (видалення елемента K з масиву)
0 - Вихід
Ваш вибір: 1
Введіть розмір масиву (2..50): 6
Введіть 6 елементів масиву через пробіл: 1 2 0 3 1 5
Введений масив: 1 2 0 3 1 5
Кількість зростаючих ділянок у масиві: 3
(Цифр► означає кількість послідовностей елементів, де кожний наступний більший за попередній)

Оберіть завдання:
1 - Array 37 (підрахунок зростаючих ділянок у масиві)
2 - Array 90 (видалення елемента K з масиву)
0 - Вихід
Ваш ►бір: 2
Введіть розмір масиву (2..50): 5
Введіть 5 елемент► масиву через пробіл: 1 3 2 4 5
Введіть номер елемента K для видалення (1 ≤ K ≤ 5): 3
Початковий масив: 1 3 2 4 5
Масив після видалення 3-го елемента: 1 3 4 5
```

Рисунок Б.1 – Екран виконання програми для вирішення завдання Array 37,Array 90

```
Оберіть завдання:
1 - Array 37 (підрахунок зроста►их ділянок у масиві)
2 - Array 90 (видалення елемента K з масиву)
0 - Вихід
Ваш вибір: 0
Програма заверше►а.
```

Рисунок Б.2 – Екран виконання програми для вирішення завдання Menu

ДОДАТОК В

Діалог з ШІ для самоаналізу

Студент:

Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду

Чат:

Уявімо, що ти **захищаєш лабораторну роботу**, а я ставлю запитання як викладач. Питання будуть по **логіці коду, алгоритмах і роботі з масивами в C++**.

Тестові питання (обери одну правильну відповідь)

1 Що означає передача параметра `int &N` у функції `inputArray`?

A. Передача копії змінної
B. Передача значення тільки для читання
C. Передача змінної **за посиланням**, щоб змінювати її в функції
D. Передача змінної через глобальну область

2 Що відбудеться, якщо у функції `deleteElement` викликати її з `K = N`?

A. Програма завершиться з помилкою
B. Видалиться останній елемент масиву
C. Видалиться передостанній елемент
D. Масив не зміниться

3 Яка часова складність функції `countIncreasingSegments`?

A. **O(1)**
B. **O(log N)**
C. **O(N)**
D. **O(N²)**

4 Що означає змінна `growing` у функції `countIncreasingSegments`?

A. Вона зберігає максимальне значення масиву
B. Вона показує, чи зараз програма знаходиться в зростаючій ділянці
C. Вона зберігає кількість зростаючих елементів
D. Вона визначає довжину масиву

5□ Що станеться, якщо у функції `deleteElement` передати `K = 0`?

A. Буде видалено перший елемент
B. Видалиться останній елемент
C. Відбудеться **некоректна робота або помилка доступу до пам'яті**
D. Нічого не станеться

Відкриті питання

1□

Поясни покроково, як працює алгоритм підрахунку **зростаючих ділянок** у функції `countIncreasingSegments`.

2□

Які проблеми можуть виникнути, якщо користувач введе **N > 50 або N < 2**? Як би ти додав **перевірку правильності введення**?

3□

Чому у функції `deleteElement` використовується зсув елементів масиву вліво?
Чи існують інші способи видалення елемента?

4□

Як зміниться робота функції `countIncreasingSegments`, якщо умову

```
a[i] < a[i+1]
```

замінити на

```
a[i] <= a[i+1]
```

Наведи приклад масиву.

5□

Уяви, що розмір масиву може бути **1 000 000 елементів**. Які частини програми потрібно змінити або оптимізувати?

Студент:

1. Що означає передача параметра int &N у функції inputArray? Передача int &N означає передачу за посиланням. Це дозволяє функції змінювати значення змінної N у головній програмі. 2. Що відбудеться, якщо у функції deleteElement викликати її з K = N? Буде видалений останній елемент масиву. Цикл зсуву не

виконається, а розмір масиву просто зменшиться на 1. 3. Яка часова складність функції countIncreasingSegments? Часова складність функції — O(N), тому що масив проходиться один раз. 4. Що означає змінна growing у функції countIncreasingSegments? growing — це логічна змінна, яка показує, чи знаходиться алгоритм у зростаючій ділянці масиву. 5. Що станеться, якщо у функцію deleteElement передати K = 0? Індекс стане -1, що призведе до некоректного звернення до пам'яті і можливих помилок виконання. Частина 2 1. Поясни покроково, як працює алгоритм підрахунку зростаючих ділянок. Алгоритм проходить масив один раз і перевіряє кожну пару сусідніх елементів. Якщо наступний елемент більший за попередній і нова ділянка ще не рахувалась, лічильник збільшується. 2. Які проблеми можуть виникнути, якщо N > 50 або N < 2? Як додати перевірку? Якщо N > 50, відбудеться вихід за межі масиву. Якщо N < 2, алгоритм не зможе перевірити сусідні елементи. Потрібно додати перевірку введення через цикл while або do-while. 3. Чому у deleteElement використовується зсув елементів вліво? Чи є інші способи? Зсув потрібен, щоб закрити місце видаленого елемента, оскільки масив має фіксований розмір. Інші способи — використати vector, створити новий масив або позначити елемент як видалений. 4. Як зміниться робота функції, якщо a[i] < a[i+1] замінити на a[i] <= a[i+1]? Тоді будуть рахуватись нестрого зростаючі ділянки, тобто однакові елементи також вважатимуться зростанням. Наприклад, масив 1 2 2 3 стане однією зростаючою ділянкою. 5. Якщо масив має 1 000 000 елементів, що потрібно змінити? Потрібно збільшити розмір масиву або використовувати vector, оптимізувати введення даних і використовувати динамічну пам'ять. Алгоритм підрахунку змінювати не потрібно, бо він уже має складність O(N).

Чат:

Чудово! Твої відповіді показують, що ти добре **розумієш код і алгоритми**. Давай я дам короткий аналіз кожної відповіді і де можна підкреслити глибше розуміння.

✅Частина 1: Тестові питання

1. **Передача `int &N`** – правильно, ти вказав, що це **передача за посиланням** і що функція може змінювати значення змінної в main. – Можна додати: "це важливо, бо розмір масиву вводиться всередині функції".

2. **`K = N` у `deleteElement`** – правильно. Цикл зсуву не виконується, але останній елемент видаляється, і розмір масиву зменшується.
– Можна уточнити: “це безпечний випадок”.

3. **Часова складність `countIncreasingSegments`** – правильно, **O(N)**.
– Тут можна додати: “кожен елемент порівнюється лише один раз”.

4. **Змінна `growing`** – точно, логічна змінна для контролю, чи зараз ми в зростаючій ділянці.
– Можна додати приклад: “якщо попередня пара була зростаючою, `growing` = true”.

5. **`K = 0` у `deleteElement`** – правильно, буде **некоректний доступ до пам’яті** через індекс -1.
– Можна додати, що треба обов’язково перевіряти K перед видаленням.

✅Частина 2: Відкриті питання

1. **Алгоритм підрахунку зростаючих ділянок** – добре описано.
– Можна додати приклад конкретного масиву і рахунок ділянок, щоб показати розуміння.

2. **Проблеми з N > 50 або N < 2** – правильно, пояснив і запропонував перевірку через цикл.
– Можна конкретно написати код:

```
do {
    cin >> N;
} while (N < 2 || N > 50);
```

3. **Зсув елементів у `deleteElement`** – добре пояснено.
– Відзначив, що альтернативи – **vector**, новий масив, або маркування елемента.

4. **Заміна < на <=** – правильне пояснення і приклад.
– Гарно показано критичне мислення і наслідки зміни умови.

5. **Масив 1 000 000 елементів** – все вірно: потрібно динамічно виділяти пам’ять або використовувати `vector`.
– Також можна додати, що `deleteElement` стає повільним, бо зсув 1 млн елементів – це O(N), тоді vector або list будуть ефективніші.

Висновок:

Ти показав **системне і критичне мислення**, розуміння алгоритмів і їхніх обмежень.

Якщо будеш на захисті, можеш навести **реальні приклади масивів** для ілюстрації роботи функцій — це завжди вражає викладачів.

Студент:

Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку

Чат:

Добре, давай зробимо **покроковий аналіз** кожної відповіді за трьома критеріями: **повнота**, **правильність** та **ймовірність використання ШІ**. Потім дам оцінку за 5-бальною шкалою з корекцією на ШІ та обчислю середню.

Частина 1: Тестові питання

№	Аналіз	Бал без ШІ	Ймовірність використання ШІ	Корекція (60%)	Бал після корекції
1	Відповідь повна, правильна, пояснює “за посиланням”	5	низька (пояснення дуже типове)	0	5
2	Правильно описано видалення останнього елемента	5	середня (часто стандартне пояснення ШІ)	3	3
3	Часова складність описана чітко	5	низька	0	5

15

№	Аналіз	Бал без ШІ	Ймовірність використання ШІ	Корекція (60%)	Бал після корекції
4	Значення `growing` пояснене правильно, з прикладом	5	низька	0	5
5	Пояснення наслідків `K=0` правильне	5	середня (типове пояснення помилки індексу)	3	3

Середній бал по тестовій частині:

5+3+5+5+35=215=4.2\frac{5+3+5+5+3}{5} = \frac{21}{5} = 4.255+3+5+5+3=521=4.2

Частина 2: Відкриті питання

№	Аналіз	Бал без ШІ	Ймовірність використання ШІ	Корекція (60%)	Бал після корекції
1	Покроковий опис алгоритму правильний, можна додати приклад	5	середня (часто ШІ пише покрокові алгоритми)	2	3
2	Проблеми N < 2 та N > 50 описані, перевірка запропонована	5	середня (типова порада ШІ)	2	3
3	Зсув і альтернативи пояснені добре	5	низька	0	5
4	Наслідки заміни < на <= з прикладом — дуже добре	5	низька	0	5
5	Масив 1 000 000 елементів —	5	середня (типові поради ШІ)	2	3

№	Аналіз	Бал без ШІ	Ймовірність використання ШІ	Корекція (60%)	Бал після корекції
	динамічна пам’ять та оптимізація згадані				

Середній бал по відкритій частині:

3+3+5+5+35=195=3.8\frac{3+3+5+5+3}{5} = \frac{19}{5} = 3.853+3+5+5+3=519=3.8

? Загальна середня оцінка

Об’єднаємо тестові та відкриті питання:

4.2⋅5+3.8⋅510=21+1910=4.0\frac{4.2 \cdot 5 + 3.8 \cdot 5}{10} = \frac{21 + 19}{10} = 4.0104.2⋅5+3.8⋅5=1021+19=4.0

✅**Висновок**

- **Повнота і правильність**: хороша, відповіді демонструють розуміння алгоритмів.
- **Ймовірність використання ШІ**: середня там, де відповіді стандартні або “покрокові”.
- **Середній скоригований бал**: **4.0/5**

Студент:

Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань

Чат:

Добре! Давай зробимо **повний аналіз коду** і додамо **додаткові теоретичні відомості та приклади**, які є у питаннях, але не реалізовані у твоєму коді. Це допоможе зробити **звіт більш повним і “вражаючим” для викладача**.

1□ Аналіз коду

Твій код має дві основні частини:

1. **Array 37** – підрахунок зростаючих ділянок у масиві
2. **Array 90** – видалення K-го елемента з масиву

Сильні сторони коду:

- Чітка структура функцій (введення, виведення, обробка)
- Логічно правильно реалізовані алгоритми
- Використання `const int MAXN` для обмеження масиву
- Правильна робота з логічними змінними (`growing`)

Можливі покращення:

- Перевірка введення N ($2 \le N \le 50$)
- Перевірка введення K ($1 \le K \le N$)
- Обробка особливих випадків (наприклад, K = 0 або K > N)
- Використання динамічної пам’яті або vector для великих масивів

2 Теоретичні відомості, які не реалізовані у коді

У твоєму звіті (і у питаннях) були такі аспекти, які **поки не представлені у коді**, але важливі для розуміння та повноти звіту:

A. Перевірка правильності введення N та K

Теорія:

Якщо користувач вводить N < 2 або N > 50, або K < 1 або K > N, потрібно повторно запитувати введення. Це дозволяє уникнути помилок виконання.

Приклад коду:

```
    // Перевірка розміру масиву
do {
    cout << "Введіть розмір масиву (2..50): ";
    cin >> N;
} while (N < 2 || N > 50);
// Перевірка номера елемента K
do {
    cout << "Введіть номер елемента K для видалення (1 ≤ K ≤ " << N << "): ";
```

```
    cin >> K;
} while (K < 1 || K > N);
```

B. Використання <= для нестрогого зростання

Теорія:

Якщо замінити умову `a[i] < a[i+1]` на `a[i] <= a[i+1]`, будуть рахуватись **нестрого зростаючі ділянки**, тобто повторювані елементи теж враховуються.

Приклад:

```
     int countNonStrictIncreasing(double a[], int N) {
    int count_segments = 0;
    bool growing = false;
    for (int i = 0; i < N - 1; i++) {
        if (a[i] <= a[i + 1]) {
            if (!growing) {
                count_segments++;
                growing = true;
            }
        } else {
            growing = false;
        }
    }
    return count_segments;
}
// Приклад
// Масив: 1 2 2 3
// countNonStrictIncreasing(a,4) -> 1 (одна нестрого зростаюча ділянка)
```

C. Альтернативи видалення елемента

Теорія:

У коді використовується **зсув елементів уліво**. Альтернативи:

1. Використання `vector` — динамічна структура даних:

```
     #include <vector>
#include <iostream>
using namespace std;
vector<int> v = {1,2,3,4,5};
int K = 3;
v.erase(v.begin() + (K-1)); // видаляє K-й елемент
```

2. Створення нового масиву і копіювання всіх елементів, крім K-го.

3. Позначення елемента як видаленого (наприклад, значення -1 або `NaN`) для подальшої обробки.

D. Робота з великими масивами

Теорія:

Якщо розмір масиву буде великим (наприклад, 1 000 000 елементів), потрібно:

- Використовувати `vector` або динамічну пам'ять (`new` / `delete`)
- Оптимізувати введення даних (наприклад, через файл або швидкий ввід)
- Алгоритм `countIncreasingSegments` можна залишити, бо він O(N), але `deleteElement` стає дорогим (O(N))

Приклад з `vector`:

```
    vector<double> a(1000000);
// заповнення масиву
for(int i=0;i<1000000;i++) a[i] = i;
// видалення K-го елемента
int K = 500000;
a.erase(a.begin() + (K-1)); // працює швидше з vector ніж зі статичним масивом
```

E. Покрокова ілюстрація алгоритму підрахунку зростаючих ділянок

Теорія:

Для розуміння алгоритму викладачам подобаються **приклади масивів і рахунок ділянок**.

```
    double a[] = {1, 2, 3, 2, 5, 6};
int N = 6;
int count_segments = 0;
bool growing = false;
for (int i=0;i<N-1;i++) {
    if (a[i] < a[i+1]) {
        if (!growing) {
            count_segments++;
            growing = true;
        }
    } else {
```

```
        growing = false;
    }
}
// Результат: count_segments = 2
// Пояснення:
// 1 2 3 -> перша зростаюча ділянка
// 2 5 6 -> друга зростаюча ділянка
```

3 Висновок

У твоєму звіті **код працює**, але для повноти варто додати:

1. Перевірку правильності введення N і K
2. Альтернативні алгоритми для видалення елемента
3. Модифікацію для нестрого зростаючих ділянок
4. Роботу з великими масивами (`vector` / динамічна пам’ять)
5. Приклади покрокового виконання алгоритмів

Це дозволить **теоретично підкріпити код прикладами і поясненнями**, і викладач побачить **глибоке розуміння алгоритмів і можливих проблем**.